

计算机组成原理

Computer Organization & Architecture

MS Roslyn

2026 年 8 月 4 日

摘要

本教程系统讲解计算机组成原理的核心概念，从底层硬件到上层接口逐层展开：数据在机器中的表示与运算、数字逻辑基础、指令系统设计、处理器数据通路与控制、流水线与指令级并行、存储层次与缓存、I/O 系统与总线架构。面向计算机专业学生及自学者，兼顾理论基础与工程直觉。

目录

1	计算机系统概述	6
1.1	计算机是什么	6
1.2	存储程序原理	6
1.3	计算机层次结构	6
1.4	性能评价指标	7
2	数据表示与运算	8
2.1	数制与编码	8
2.2	整数表示	8
2.2.1	无符号整数	8
2.2.2	原码 (Sign-Magnitude)	8
2.2.3	补码 (Two's Complement)	8
2.2.4	移码 (Biased Notation)	9
2.3	整数运算	9
2.3.1	加法器	9
2.3.2	溢出检测	9
2.4	浮点数表示: IEEE 754	9
2.5	浮点运算	10
2.6	字符编码	10
3	数字逻辑基础	11
3.1	布尔代数与门电路	11
3.2	逻辑表达式化简	12
3.2.1	代数化简	12
3.2.2	卡诺图 (Karnaugh Map)	12
3.3	组合逻辑电路: 加法器	12
3.3.1	半加器 (Half Adder)	12
3.3.2	全加器 (Full Adder)	12
3.3.3	行波进位加法器 (Ripple Carry Adder)	13
3.3.4	超前进位加法器 (Carry Lookahead Adder)	13
3.3.5	溢出检测	14
3.4	组合逻辑电路: 数据选择器与译码器	14
3.4.1	多路选择器 (Multiplexer, MUX)	14
3.4.2	译码器 (Decoder)	15
3.5	算术逻辑单元 (ALU)	16

3.6	时序逻辑电路	17
3.6.1	SR 锁存器	17
3.6.2	D 触发器 (D Flip-Flop)	18
3.6.3	n 位寄存器	19
3.7	有限状态机 (FSM)	19
4	指令系统与汇编	20
4.1	指令集架构 (ISA)	20
4.2	RISC 与 CISC	20
4.3	MIPS 指令集	20
4.3.1	MIPS 寄存器约定	21
4.3.2	三种指令格式	21
4.3.3	寻址方式	22
4.4	过程调用约定	22
4.5	x86 指令集简介	22
5	处理器：数据通路	23
5.1	单周期 CPU 概述	23
5.2	数据通路组件	23
5.3	基本指令的执行	23
5.3.1	R 型指令	23
5.3.2	I 型: lw	23
5.3.3	I 型: sw	24
5.3.4	I 型: beq	24
5.4	完整单周期数据通路	24
5.5	关键路径与性能	24
6	处理器：控制器	25
6.1	控制器的功能	25
6.2	控制信号定义	25
6.3	ALU 控制	25
6.4	控制器实现方式	26
6.4.1	硬布线控制器 (Hardwired Control)	26
6.4.2	微程序控制器 (Microprogrammed Control)	26
6.5	单周期 vs 多周期 CPU	26

7	流水线与指令级并行	28
7.1	流水线原理	28
7.2	经典五级流水线	28
7.3	流水线冒险 (Pipeline Hazards)	28
7.3.1	结构冒险 (Structural Hazard)	28
7.3.2	数据冒险 (Data Hazard)	28
7.3.3	控制冒险 (Control Hazard)	29
7.4	分支预测	29
7.5	超标量与乱序执行	29
7.6	流水线的性能极限	30
8	存储层次与缓存	31
8.1	存储层次概述	31
8.2	局部性原理	31
8.3	SDRAM 与 DRAM	31
8.4	缓存 (Cache)	31
8.4.1	直接映射缓存 (Direct-Mapped)	31
8.4.2	组相联缓存 (Set-Associative)	32
8.4.3	缓存地址划分	32
8.4.4	缓存缺失与写策略	32
8.4.5	缓存性能	32
8.5	虚拟内存	32
8.5.1	TLB (快表)	33
8.5.2	多级页表	33
8.6	存储层次设计原则	33
9	I/O 系统与总线	34
9.1	I/O 系统的层次	34
9.2	内存映射 I/O vs 独立 I/O	34
9.3	I/O 同步方式	34
9.3.1	轮询 (Polling)	34
9.3.2	中断 (Interrupt)	34
9.3.3	直接存储器访问 (DMA)	35
9.4	总线	35
9.4.1	总线分类	35
9.4.2	现代总线: PCIe	35

9.5	磁盘与固态存储	35
9.6	多核与并行 I/O	36
9.7	系统总线架构总结	36

1 计算机系统概述

1.1 计算机是什么

计算机 (Computer) 是由处理器、存储器、输入输出设备组成的通用信息处理系统。它的核心功能可以用冯·诺依曼模型概括：

定义 1.1 (冯·诺依曼体系结构). 计算机由五大部件构成：

- 运算器 (*Arithmetic Logic Unit, ALU*): 执行算术与逻辑运算
- 控制器 (*Control Unit, CU*): 取指令、译码、控制数据流向
- 存储器 (*Memory*): 存放程序与数据
- 输入设备 (*Input*): 键盘、鼠标、传感器等
- 输出设备 (*Output*): 显示器、打印机、网络接口等

运算器与控制器合称中央处理器 (*CPU*)；程序与数据统一以二进制形式存储在同一个存储器中——这是冯·诺依曼体系的核心特征：**存储程序** (*Stored Program*)。

$$\text{计算机} = \text{CPU} + \text{存储器} + \text{I/O} \quad (1)$$

1.2 存储程序原理

冯·诺依曼模型的关键在于**指令和数据存放在同一个存储器中**，CPU 按顺序逐条取出并执行。这一设计使得计算机成为“通用机器”——更换程序即可完成不同任务，无需改动硬件。

$$\begin{aligned} & \text{Fetch} \rightarrow \text{Decode} \rightarrow \text{Execute} \\ & \text{取指: } \text{PC} \rightarrow \text{MAR}, M[\text{MAR}] \rightarrow \text{MDR} \\ & \text{译码: 控制器解析操作码与地址码} \\ & \text{执行: ALU 运算 / 访存 / 跳转} \end{aligned} \quad (2)$$

1.3 计算机层次结构

现代计算机是一个层次化系统，每一层对上一层隐藏细节：

- 应用层：高级语言程序 (C/Java/Python)

- 系统软件层：编译器、操作系统
- 指令集层：ISA——软件与硬件的接口
- 微体系结构层：处理器实现（流水线、缓存、乱序执行）
- 数字逻辑层：门电路、寄存器、加法器
- 物理器件层：晶体管、导线、集成电路

”计算机组成原理”关注的是指令集层到数字逻辑层之间的内容——硬件如何执行指令。

1.4 性能评价指标

表 1: 计算机性能核心公式

指标	公式	含义
CPU 执行时间	$T = \text{指令数} \times \text{CPI} \times \text{时钟周期}$	运行一段程序的总时间
CPI	每条指令平均时钟周期数	效率指标
MIPS	百万指令/秒	吞吐量指标
时钟频率	$f = 1/T_{\text{clk}}$	CPU 主频
Amdahl 定律	$S = \frac{1}{(1-f)+f/n}$	并行加速上限

2 数据表示与运算

2.1 数制与编码

计算机中所有信息都以二进制（0/1）存储。本章讨论如何用二进制表示整数、浮点数、字符。

表 2: 常用数制对比

进制	基数	数字符号
二进制 (Binary)	2	0, 1
八进制 (Octal)	8	0-7
十进制 (Decimal)	10	0-9
十六进制 (Hex)	16	0-9, A-F

$$(N)_r = \sum_{i=-m}^{n-1} d_i \cdot r^i \quad (3)$$

2.2 整数表示

2.2.1 无符号整数

$$U(X) = \sum_{i=0}^{n-1} x_i \cdot 2^i, \quad \text{范围: } [0, 2^n - 1] \quad (4)$$

2.2.2 原码 (Sign-Magnitude)

最高位为符号位（0 正 / 1 负），其余位表示绝对值。存在 +0（0000）和 -0（1000）两个零。

2.2.3 补码 (Two's Complement)

现代计算机统一采用补码表示有符号整数——优势是加减法统一，不存在双零。

定义 2.1 (补码). n 位有符号整数 X 的补码值为：

$$T(X) = -x_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} x_i \cdot 2^i$$

范围： $[-2^{n-1}, 2^{n-1} - 1]$ （如 8 位为 $[-128, 127]$ ）。

示例 2.1 (补码转换). 8 位补码 1000 0001 的值:

$$-1 \cdot 2^7 + 1 \cdot 2^0 = -128 + 1 = -127$$

取反加一法 (快速转换): 原码 $\rightarrow$ 逐位取反 $\rightarrow$ 末位加一。

2.2.4 移码 (Biased Notation)

用于浮点数阶码。 $E_{\text{biased}} = E + \text{Bias}$, 单精度 $\text{Bias} = 127$, 双精度 $\text{Bias} = 1023$ 。

2.3 整数运算

2.3.1 加法器

$$\text{全加器: } S = A \oplus B \oplus C_{\text{in}}, \quad C_{\text{out}} = AB + (A \oplus B)C_{\text{in}}$$

行波进位加法器 (Ripple Carry Adder): n 个全加器串联, 延迟 $O(n)$ 。超前进位加法器 (Carry Lookahead Adder): 并行计算进位, 延迟 $O(\log n)$ 。

2.3.2 溢出检测

- 同号相加得异号 $\rightarrow$ 溢出
- 补码: $A_{n-1} = B_{n-1} = 0$ 且 $S_{n-1} = 1$; 或 $A_{n-1} = B_{n-1} = 1$ 且 $S_{n-1} = 0$
- 硬件判断: $O = C_n \oplus C_{n-1}$

2.4 浮点数表示: IEEE 754

定义 2.2 (IEEE 754 单精度浮点数). 32 位: 1 位符号 S + 8 位移码阶码 E + 23 位尾数 M (隐含 1):

$$(-1)^S \times (1.M) \times 2^{E-127}$$

表 3: IEEE 754 格式参数

精度	总位数	阶码位	尾数位
单精度 (float)	32	8	23
双精度 (double)	64	11	52
扩展双精度	80	15	64

- 规格化数: $E \neq 0$ 且 $E \neq 255$, 尾数隐含 1
- 非规格化数 (denormal): $E = 0$, 尾数隐含 0, 用于表示极小数
- 无穷大: $E = 255$, $M = 0$
- NaN: $E = 255$, $M \neq 0$

舍入模式: 向偶数舍入 (Round to Nearest Even) (5)

2.5 浮点运算

加法: 对阶 $\rightarrow$ 尾数相加 $\rightarrow$ 规格化 $\rightarrow$ 舍入
乘法: 阶码相加 $\times$ 尾数相乘 $\rightarrow$ 规格化 $\rightarrow$ 舍入 (6)

2.6 字符编码

- **ASCII**: 7 位 (0-127), 扩展 ASCII 为 8 位
- **Unicode**: 统一的全球字符集
- **UTF-8**: 变长编码, ASCII 兼容 (1-4 字节), 互联网标准
- **GBK / GB 2312**: 中文编码标准

3 数字逻辑基础

3.1 布尔代数与门电路

数字电路的基础是布尔代数，变量取值 $\{0, 1\}$ ，三种基本运算：

$$\text{与 (AND)} : A \cdot B = 1 \iff A = 1 \wedge B = 1$$

$$\text{或 (OR)} : A + B = 1 \iff A = 1 \vee B = 1 \quad (7)$$

$$\text{非 (NOT)} : \bar{A} = 1 \iff A = 0$$

由这三种基本运算可组合出所有逻辑函数。下表给出常用门电路的真值表：

表 4: 基本门电路真值表

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

门电路在电路图中的标准符号如下：

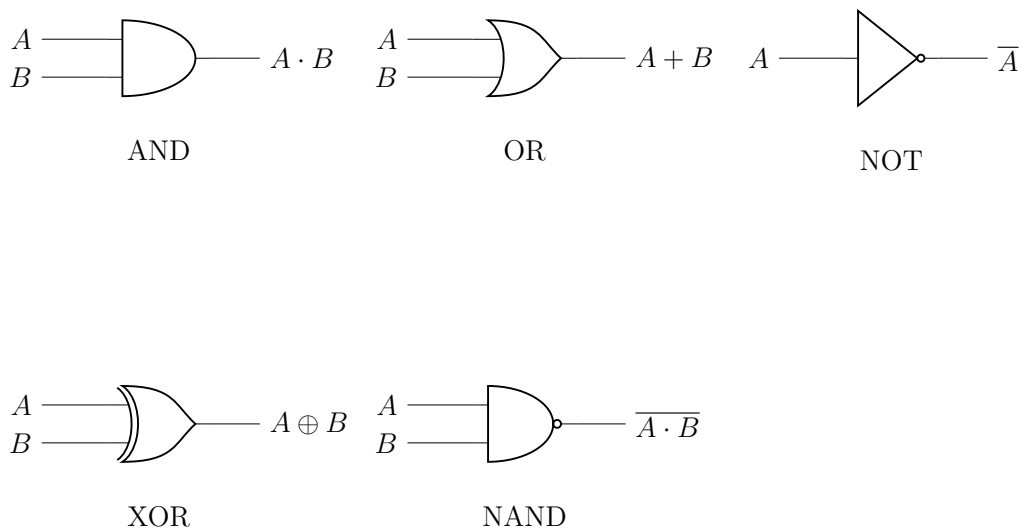


图 1: 基本门电路符号

NAND (与非门) 是“万能门”——仅用 NAND 可实现 AND、OR、NOT 等全部逻辑，是数字集成电路最常见的实现方式。

3.2 逻辑表达式化简

3.2.1 代数化简

结合律、分配律、吸收律、德·摩根定律：

$$\overline{A \cdot B} = \overline{A} + \overline{B}, \quad \overline{A + B} = \overline{A} \cdot \overline{B} \quad (8)$$

德·摩根定律的电路等价性：一个 NAND 门等效于反相输入后的 OR 门；一个 NOR 门等效于反相输入后的 AND 门。这为 CMOS 实现提供了理论基础。

3.2.2 卡诺图 (Karnaugh Map)

最多 4-6 变量的图形化化简工具：将真值表画在网格上，圈出相邻的 1 形成最小项合并。

3.3 组合逻辑电路：加法器

加法器是从门电路到 CPU 运算器的桥梁。先从半加器开始。

3.3.1 半加器 (Half Adder)

半加器将两个比特相加，输出和位 (Sum) 与进位 (Carry)：

$$S = A \oplus B, \quad C_{\text{out}} = A \cdot B \quad (9)$$

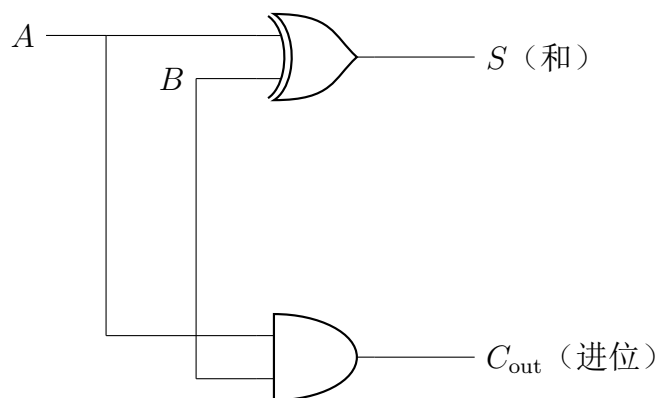


图 2: 半加器：一个 XOR 和一个 AND

3.3.2 全加器 (Full Adder)

全加器加上了来自低位的进位 C_{in} ：

$$\begin{aligned}
 S &= A \oplus B \oplus C_{\text{in}} \\
 C_{\text{out}} &= AB + (A \oplus B)C_{\text{in}}
 \end{aligned}
 \tag{10}$$

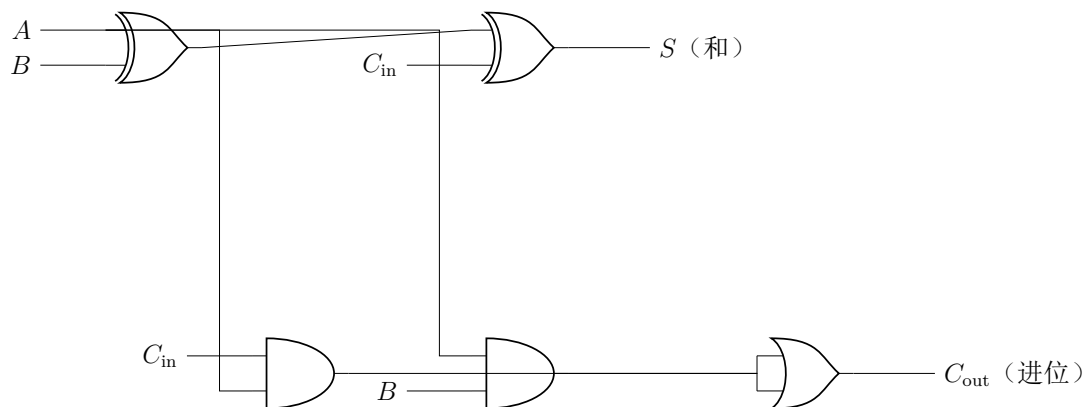
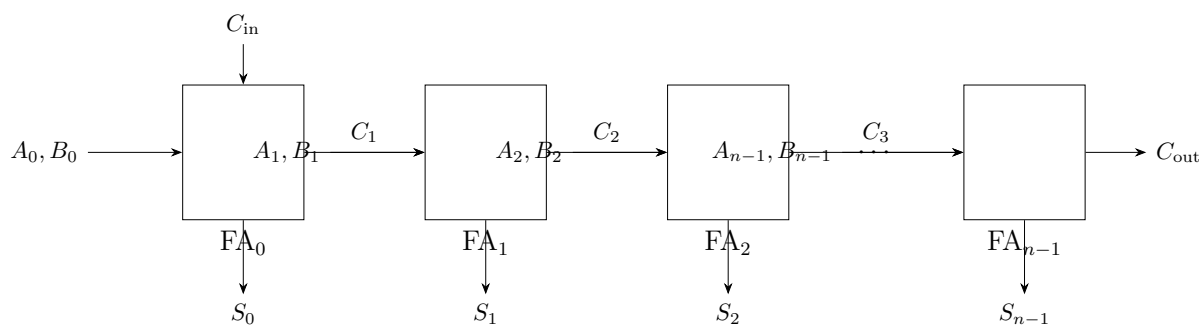


图 3: 全加器: 2 个半加器 + 1 个 OR 门

3.3.3 行波进位加法器 (Ripple Carry Adder)

将 n 个全加器串联，进位像“波浪”一样从低位向高位传播：

图 4: n 位行波进位加法器：进位逐级传播，延迟 $O(n)$

行波进位加法器的延迟为 $O(n)$ ——最高位必须等待所有低位进位算出。下一节介绍加速方案。

3.3.4 超前进位加法器 (Carry Lookahead Adder)

定义传播信号 $P_i = A_i \oplus B_i$ 和生成信号 $G_i = A_i \cdot B_i$ ，则进位的递推关系可展开为超前进位逻辑：

$$\begin{aligned}
C_1 &= G_0 + P_0 C_0 \\
C_2 &= G_1 + P_1 G_0 + P_1 P_0 C_0 \\
C_3 &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \\
C_4 &= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0
\end{aligned} \tag{11}$$

每级进位由独立的“超前进位单元”（Carry Lookahead Unit, CLU）并行产生，延迟 $O(\log n)$ 。实际设计中常用 4 位块 CLA 级联（Ripple between blocks）。

3.3.5 溢出检测

- 同号相加得异号 $\rightarrow$ 溢出
- $O = C_n \oplus C_{n-1}$: 最高位和次高位的进位不同即溢出

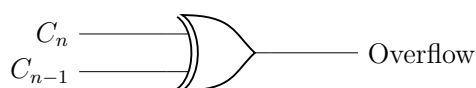


图 5: 溢出检测电路

3.4 组合逻辑电路：数据选择器与译码器

3.4.1 多路选择器（Multiplexer, MUX）

2^n 路输入 + n 位选择信号 $\rightarrow$ 1 路输出。下面是 2-to-1 MUX 的门级实现：

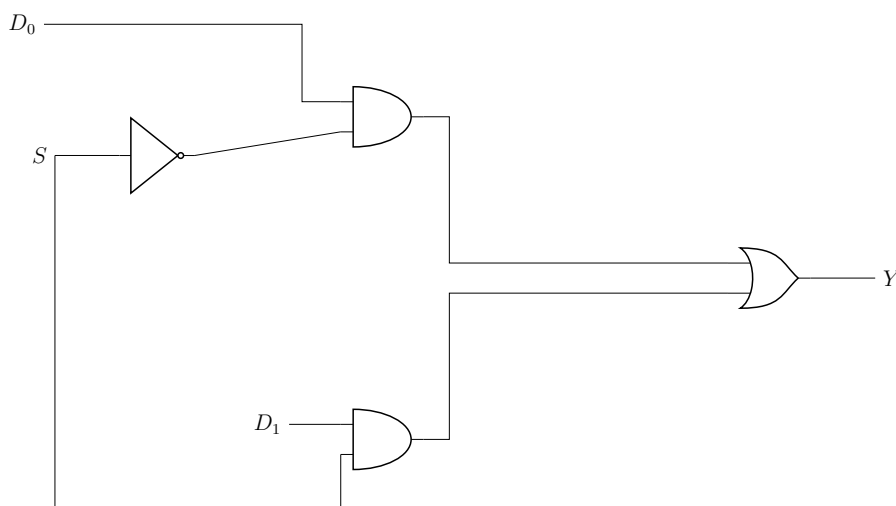


图 6: 2-to-1 MUX 门级实现： $Y = \bar{S} \cdot D_0 + S \cdot D_1$

$$Y = \overline{S} \cdot D_0 + S \cdot D_1 \quad (12)$$

抽象层面上, MUX 常用梯形符号表示 (宽边为输入, 窄边为输出, 底部为选择信号):

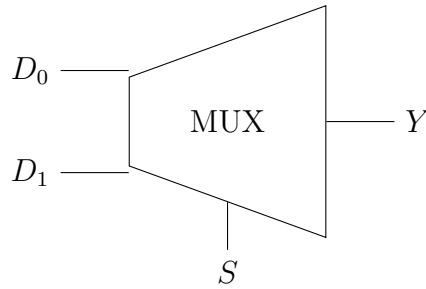


图 7: MUX 抽象符号

3.4.2 译码器 (Decoder)

n 位输入 $\rightarrow 2^n$ 位输出, 任意时刻仅有 1 位有效。2-to-4 译码器示意 (4 条垂直“总线”分别携带 $A_1, \overline{A_1}, A_0, \overline{A_0}$, 各 AND 门横向接入避免走线交叉):

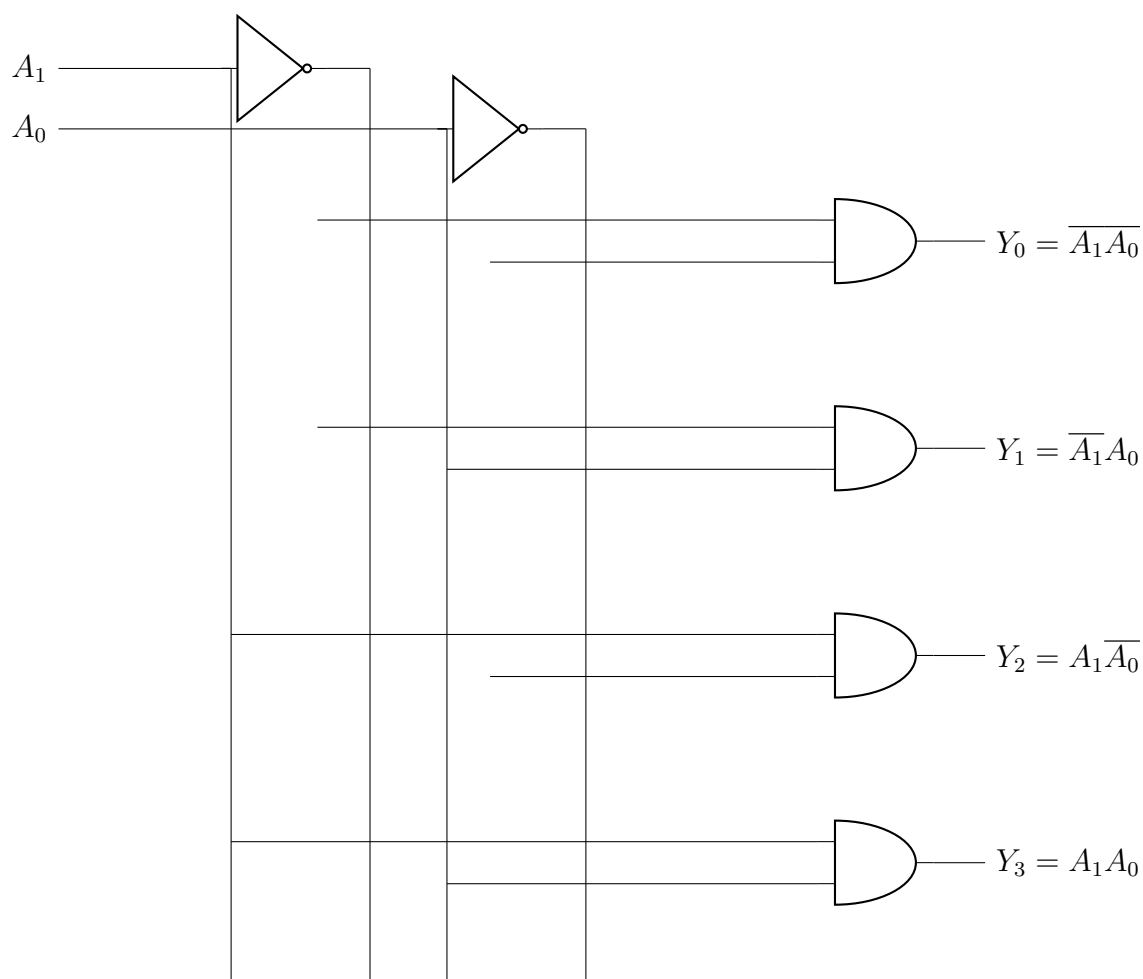


图 8: 2-to-4 译码器

3.5 算术逻辑单元 (ALU)

ALU 是 CPU 的核心运算部件。下图展示一个支持 AND、OR、ADD 三种运算的 1-bit ALU:

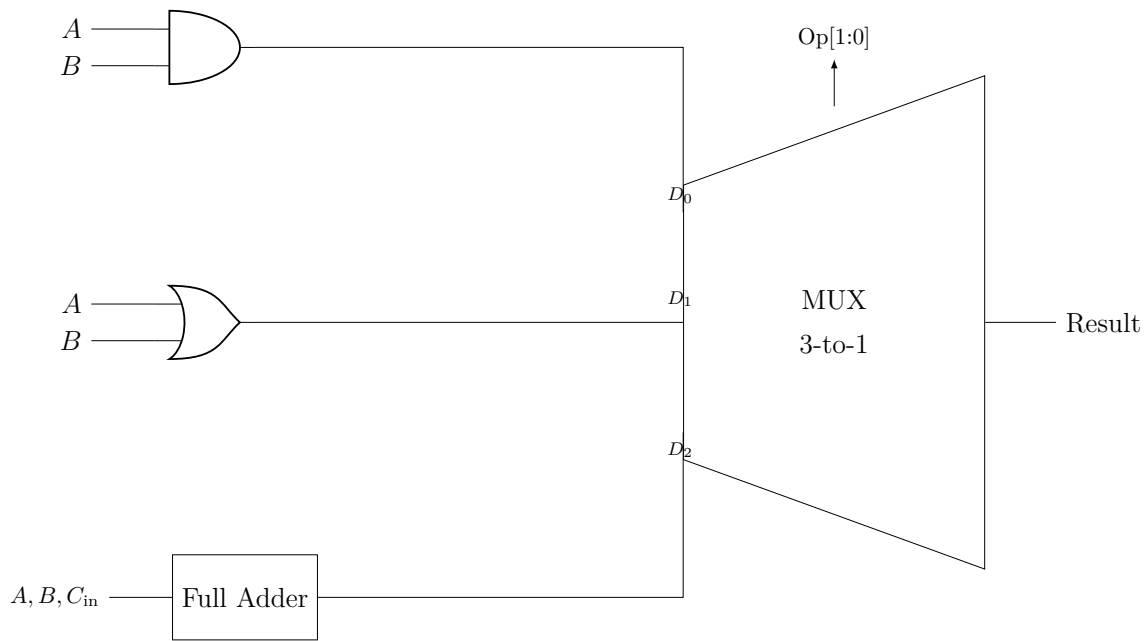


图 9: 1-bit ALU 结构

$$\text{Result} = \begin{cases} A \wedge B, & \text{Op} = 00 \\ A \vee B, & \text{Op} = 01 \\ A + B + C_{\text{in}}, & \text{Op} = 10 \end{cases} \quad (13)$$

32 位 ALU 由 32 个 1-bit ALU 并行构成，进位在各位之间传递（行波或超前进位）。

3.6 时序逻辑电路

时序逻辑电路的输出取决于当前输入 和历史状态（有记忆）。

3.6.1 SR 锁存器

SR 锁存器是最基本的存储单元，用两个 NOR 门交叉耦合构成：

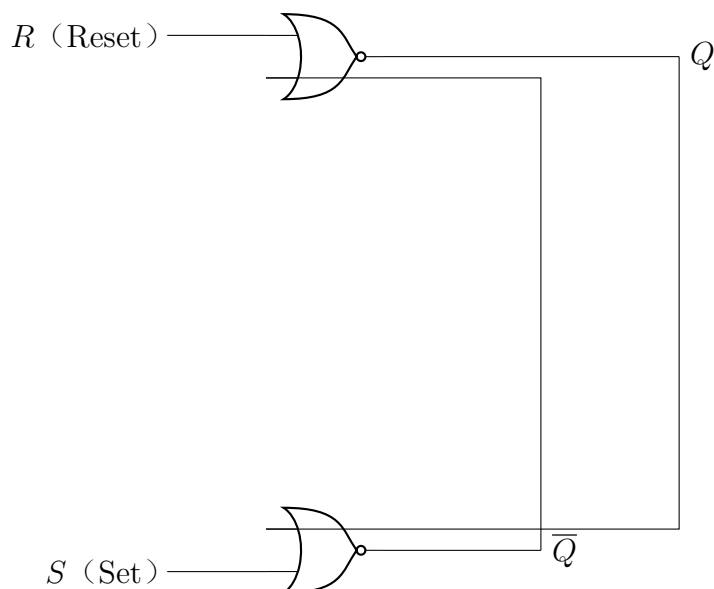


图 10: SR 锁存器 (NOR 实现)

- $S = 0, R = 0$: 保持原值
- $S = 1, R = 0$: 置位 ($Q = 1$)
- $S = 0, R = 1$: 复位 ($Q = 0$)
- $S = 1, R = 1$: 非法 ($Q = \bar{Q} = 0$, 不稳定)

3.6.2 D 触发器 (D Flip-Flop)

SR 锁存器不方便定时, D 触发器是同步的 1-bit 存储单元, 用时钟边沿 (三角形符号) 采样:

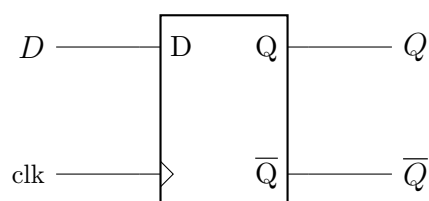


图 11: D 触发器

$$Q_{\text{next}} = D \quad (\text{时钟上升沿采样}) \quad (14)$$

D 触发器在时钟边沿采样 D 并更新 Q , 其余时间 Q 不变。 n 个 D 触发器并联即构成 n 位寄存器。

3.6.3 n 位寄存器

n 个 D 触发器共享时钟信号，同时输入/输出 n 位数据：

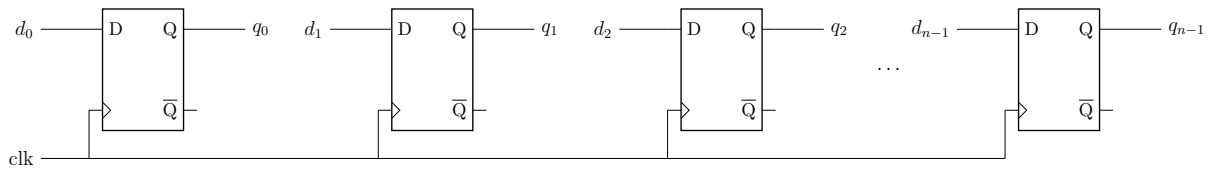


图 12: n 位寄存器: n 个 D 触发器共享时钟

3.7 有限状态机 (FSM)

定义 3.1 (FSM). 一个时序逻辑系统可抽象为有限状态机：

$$M = (S, I, O, \delta, \lambda, s_0) \quad (15)$$

S : 有限状态集, I : 输入集, O : 输出集, δ : 状态转移函数, λ : 输出函数, s_0 : 初始状态。

- **Moore 状态机:** 输出仅取决于当前状态
- **Mealy 状态机:** 输出取决于当前状态和输入

控制器就是 FSM 的经典应用：状态是各执行阶段（取指、译码、执行、访存、写回），输入是指令操作码和条件标志，输出是各数据通路组件的控制信号。

4 指令系统与汇编

4.1 指令集架构（ISA）

指令集架构（Instruction Set Architecture, ISA）是软件与硬件的接口。它定义了：

- 寄存器组：名称、数量、位宽
- 指令格式：操作码 + 操作数的编码方式
- 寻址方式：如何定位操作数
- 数据类型：支持的整数/浮点数宽
- I/O 模型：内存映射/独立 I/O

4.2 RISC 与 CISC

表 5: RISC 与 CISC 对比		
特征	RISC	CISC
指令长度	定长（如 32-bit）	变长（1-15 字节）
指令数量	少（约 100 条）	多（约 300+ 条）
寻址方式	简单（Load/Store 架构）	复杂（可直接访存运算）
典型代表	MIPS、ARM、RISC-V	x86、VAX
硬件设计	简洁，易流水	复杂，需微程序
编译负担	重（编译器优化关键）	轻（硬件承担更多）

4.3 MIPS 指令集

MIPS 是经典 RISC 架构，32 个 32 位通用寄存器（\$0 始终为 0），Load/Store 架构。

4.3.1 MIPS 寄存器约定

表 6: MIPS 寄存器用途

寄存器号	名称	用途
\$0	\$zero	恒为 0
\$1	\$at	汇编器保留
\$2-\$3	\$v0-\$v1	函数返回值
\$4-\$7	\$a0-\$a3	函数参数
\$8-\$15	\$t0-\$t7	临时变量
\$16-\$23	\$s0-\$s7	保存变量（需恢复）
\$29	\$sp	栈指针
\$31	\$ra	返回地址

4.3.2 三种指令格式

- **R 型**（寄存器型）：op(6) + rs(5) + rt(5) + rd(5) + shamt(5) + funct(6)
- **I 型**（立即数型）：op(6) + rs(5) + rt(5) + immediate(16)
- **J 型**（跳转型）：op(6) + address(26)

Listing 1: MIPS 汇编示例：数组求和
示例 4.1 (MIPS 指令示例).

```

1  # 计算 a[0] + a[1] + ... + a[9]
2  li    $t0, 0          # sum = 0
3  li    $t1, 0          # i = 0
4  la    $t2, array      # $t2 = &a[0]
5  loop:
6  lw    $t3, 0($t2)      # $t3 = a[i]
7  addu  $t0, $t0, $t3    # sum += $t3
8  addi  $t1, $t1, 1      # i++
9  addi  $t2, $t2, 4      # $t2 += 4 (next word)
10 blt   $t1, 10, loop    # if i < 10 goto loop
11 done:
12 move  $v0, $t0         # return sum

```

4.3.3 寻址方式

- 寄存器寻址: `add $1, $2, $3`
- 立即数寻址: `addi $1, $2, 100`
- 基址偏移寻址: `lw $1, 8($2)` ($\$1 = M[\$2 + 8]$)
- PC 相对寻址: `beq $1, $2, label` ($PC = PC + 4 + \text{offset} \times 4$)
- 伪直接寻址: `j addr` (跳转地址 = PC 高 4 位拼接 26 位地址 $\times 4$)

4.4 过程调用约定

MIPS 过程调用用 `jal` (jump and link) 跳转并保存返回地址到 `$ra`, 用 `jr $ra` 返回。

Listing 2: MIPS 过程调用示例

```

1  # 调用
2  jal  my_func           # $ra = PC+4; 跳转到 my_func
3
4  # 过程内部 (栈帧)
5  addi $sp, $sp, -8      # 分配栈帧
6  sw   $ra, 4($sp)       # 保存返回地址
7  sw   $s0, 0($sp)       # 保存 callee-saved 寄存器
8  # ... 过程体 ...
9  lw   $s0, 0($sp)       # 恢复 $s0
10 lw   $ra, 4($sp)       # 恢复 $ra
11 addi $sp, $sp, 8       # 释放栈帧
12 jr   $ra              # 返回

```

4.5 x86 指令集简介

x86 是典型的 CISC 架构: 变长指令 (1-15 字节), 寄存器较少 (8 个通用寄存器, x86-64 扩展到 16 个), 支持内存直接参与运算 (非 Load/Store)。x86-64 是目前桌面/服务器主流架构。

5 处理器：数据通路

5.1 单周期 CPU 概述

单周期 CPU 在一个时钟周期内完成从取指到写回的全部操作。CPI = 1，但时钟周期受最慢指令（如 `lw`）限制。

5.2 数据通路组件

表 7: 基本数据通路组件

组件	功能	控制信号
程序计数器 (PC)	存放下一条指令地址	PCWrite
指令存储器	按地址取指令	—
寄存器堆	32 个 32 位通用寄存器	RegWrite
ALU	算术/逻辑运算	ALUOp, ALUSrc
数据存储器	读写数据	MemRead, MemWrite
多路选择器 (MUX)	选择数据来源	选择信号
符号扩展单元	16-bit $\rightarrow$ 32-bit 扩展	—

5.3 基本指令的执行

5.3.1 R 型指令

$$\text{add } \$rd, \$rs, \$rt \longrightarrow \$rd = \$rs + \$rt \quad (16)$$

数据流: PC $\rightarrow$ 取指 $\rightarrow$ 译码 (读 $\$rs, \rt) $\rightarrow$ ALU $\rightarrow$ 写回 $\$rd \rightarrow$ PC+4

5.3.2 I 型: `lw`

$$\text{lw } \$rt, \text{offset}(\$rs) \longrightarrow \$rt = M[\$rs + \text{SignExt}(16\text{-bit offset})] \quad (17)$$

数据流: PC $\rightarrow$ 取指 $\rightarrow$ 译码 (读 $\$rs$) $\rightarrow$ ALU 算地址 $\rightarrow$ 访存 $\rightarrow$ 写回 $\$rt \rightarrow$ PC+4
存储器访问额外需要 ALU 用于地址计算，结果经数据存储器读出。

5.3.3 I 型：sw

$$\text{sw } \$rt, \text{offset}(\$rs) \longrightarrow M[\$rs + \text{SignExt}(16\text{-bit offset})] = \$rt \quad (18)$$

与 lw 类似，但写入方向相反——寄存器的值写入存储器而非读回。

5.3.4 I 型：beq

$$\text{beq } \$rs, \$rt, \text{label} \longrightarrow \text{if } \$rs = \$rt \text{ then } PC \leftarrow PC + 4 + \text{offset} \times 4 \quad (19)$$

ALU 做减法判断是否相等。若相等，PC 取分支目标 ($PC+4 + \text{offset} \times 4$)，否则取 $PC+4$ 。

5.4 完整单周期数据通路

将上述组件用多路选择器和总线连接，形成完整的 MIPS 单周期数据通路。控制信号由控制器根据指令操作码生成。

- 取指阶段：PC 送指令存储器，取出 32 位指令
- 译码阶段：寄存器堆读出 rs、rt 值，符号扩展立即数
- 执行阶段：ALU 执行运算（算术 / 地址计算 / 比较）
- 访存阶段：读/写数据存储器（仅 lw / sw）
- 写回阶段：结果写回寄存器堆（ALU 结果 / 存储器数据）

5.5 关键路径与性能

单周期 CPU 的时钟周期由最长路径（lw 指令：取指 + 译码 + ALU + 访存 + 写回）决定。这导致简单指令（如 add）也必须等待相同的长周期，效率很低。这是后续流水线设计的直接动机。

6 处理器：控制器

6.1 控制器的功能

控制器（Control Unit）是 CPU 的”大脑”——根据当前指令的操作码和状态标志，生成所有数据通路组件的控制信号。

$$\text{控制信号} = f(\text{opcode}, \text{funct}, \text{状态标志}) \quad (20)$$

6.2 控制信号定义

每种指令类型需要的控制信号组合：

表 8: MIPS 主要控制信号

信号	作用	受影响的指令
RegDst	选择写回目标寄存器（rt / rd）	R-type $\rightarrow$ 1, I-type $\rightarrow$ 0
RegWrite	写使能寄存器堆	R-type, lw
ALUSrc	ALU 第二个操作数来源（寄存器/立即数）	R-type $\rightarrow$ 0, I-type $\rightarrow$ 1
Mem-Read	读数据存储器	lw
MemWrite	写数据存储器	sw
Mem-toReg	写回数据来源（ALU/存储器）	R-type $\rightarrow$ 0, lw $\rightarrow$ 1
Branch	条件分支使能	beq
ALUOp	ALU 操作选择	00= 加, 01= 减, 10=funct 码决定

6.3 ALU 控制

ALU 控制接收 ALUOp（来自主控制器）和 funct 字段（仅 R 型），产生 ALU 操作选择。

表 9: ALU 操作编码

ALUOp	funct	ALU 操作	指令
00	—	加	lw, sw
01	—	减	beq
10	100000	加	add
10	100010	减	sub
10	100100	与	and
10	100101	或	or
10	101010	小于置位	slt

6.4 控制器实现方式

6.4.1 硬布线控制器 (Hardwired Control)

直接用组合逻辑门实现控制信号的真值表。速度快，但灵活性差——修改指令集需重新设计电路。常用于 RISC 架构。

$$\text{各控制信号} = \text{布尔函数}(\text{opcode}, \text{funct}, \text{状态}) \quad (21)$$

6.4.2 微程序控制器 (Microprogrammed Control)

将每条机器指令分解为若干条“微指令”序列，存于控制存储器 (ROM) 中。每步微指令产生一组控制信号。

- 优点：灵活、易于修改、支持复杂指令
- 缺点：速度较慢（需从控制存储器读取微指令）
- 适用：CISC 架构（如 x86）

6.5 单周期 vs 多周期 CPU

单周期 CPU 所有指令在一个周期内完成。多周期 CPU 每条指令划分为多个阶段，每阶段占用不同周期，不同指令可用不同周期数。

表 10: 单周期与多周期对比

特征	单周期	多周期
CPI	1	3-5（视指令而定）
时钟周期	由最慢指令决定	由最慢阶段决定
硬件重复	有（多个加法器等）	少（组件可复用）
性能	差	好于单周期
复杂度	低	中

多周期 CPU 是单周期到流水线的中间过渡设计：组件复用、不同指令不同周期，但仍有功能单元闲置问题。

7 流水线与指令级并行

7.1 流水线原理

流水线（Pipeline）将一条指令的执行分解为多个阶段，不同指令的不同阶段 ** 在时间上重叠执行 **。类比工厂流水线：每个工位（阶段）同时处理不同产品（指令）。

定义 7.1 (流水线加速比). 假设 k 级流水线执行 n 条指令（无冒险）:

$$T_{\text{流水}} = \text{填充时间} + (n - 1) \times \text{最优段延时} \quad (22)$$

加速比 $S \approx k$ （当 $n \gg k$ 时）。

7.2 经典五级流水线

MIPS 五级流水线:

- **IF** (Instruction Fetch): 取指令
- **ID** (Instruction Decode): 译码 + 读寄存器
- **EX** (Execute): ALU 运算 / 地址计算
- **MEM** (Memory Access): 访存（读/写数据存储器）
- **WB** (Write Back): 结果写回寄存器

7.3 流水线冒险 (Pipeline Hazards)

冒险是指流水线在正常推进中遇到的阻碍，分三类:

7.3.1 结构冒险 (Structural Hazard)

多个指令争用同一硬件资源。

解决方案: 增加硬件（如分离指令缓存和数据缓存——哈佛结构）、流水化功能单元。

7.3.2 数据冒险 (Data Hazard)

后续指令依赖前一条指令尚未写回的结果。

示例 7.1 (数据冒险).

1	<code>add \$1, \$2, \$3</code>	# $\$1$ 在 WB 阶段才被写入
2	<code>sub \$4, \$1, \$5</code>	# 但 <code>sub</code> 的 ID 阶段就需要 $\$1$ ——冒险!

解决方案:

1. **转发/旁路 (Forwarding / Bypassing):** 在结果”产生时刻”(EX 阶段结束)直接送给下一条指令的 EX 阶段入口, 跳过写回再读出的等待。
2. **插入气泡 (Stall / Bubble):** 在无法转发时 (如 lw 后紧接使用该数据), 暂停流水线一个周期。
3. **编译调度:** 重排指令顺序, 增大数据依赖的距离。

7.3.3 控制冒险 (Control Hazard)

分支/跳转指令改变程序流, 后续已取指令可能无效。

解决方案:

- **流水线阻塞 (Stall):** 等分支结果再取下一指令 (浪费周期)
- **分支预测 (Branch Prediction):** 预测分支走向, 提前取指
- **延迟分支 (Delayed Branch):** 分支后执行一条”延迟槽”指令

7.4 分支预测

表 11: 分支预测策略

策略	原理	准确率
静态预测	向后分支 (循环) 预测跳转	约 65%
1-bit 动态预测	记录上次跳转与否	约 80%
2-bit 饱和计数器	两次跳转成功才改预测	约 90%
GShare / 锦标赛	结合全局历史 + 局部模式	95%+

7.5 超标量与乱序执行

- **超标量 (Superscalar):** 每个周期发射多条指令。IPC > 1, 需多个功能单元并行。
- **乱序执行 (Out-of-Order Execution):** 不按程序顺序执行指令, 只保留”提交”(commit) 顺序。用重排序缓冲区 (ROB) 和寄存器重命名消除假依赖。
- **同时多线程 (SMT / Hyper-threading):** 同一核心交替执行两个线程的指令, 隐藏访存延迟。

7.6 流水线的性能极限

$$\text{实际 IPC} = \text{理想 IPC} \times (1 - \text{冒险损失})$$

影响流水线效率的主要因素：缓存缺失、数据依赖链长度、分支误预测惩罚、指令发射带宽。

8 存储层次与缓存

8.1 存储层次概述

存储器的速度、容量和价格三者存在矛盾：容量越大越慢越便宜。解决方案是**存储层次**（Memory Hierarchy）：

$$\text{寄存器} \ll \text{L1 缓存} \ll \text{L2 缓存} \ll \text{L3 缓存} \ll \text{主存} \ll \text{磁盘/SSD} \quad (23)$$

8.2 局部性原理

定义 8.1 (时间局部性). 刚访问过的数据很可能在不久的将来再次被访问（如循环中的 *sum* 变量）。

定义 8.2 (空间局部性). 如果访问了地址 x ，附近的地址 $(x+1, x+2, \dots)$ 很可能很快被访问（如顺序数组遍历）。

缓存就是利用这两个局部性，把当前“用得着”的数据从慢速存储器拷贝到快速存储器。

8.3 SDRAM 与 DRAM

- **DRAM**（动态随机存取存储器）：存储单元为电容，需要定期刷新。密度大但慢。HBM（高带宽存储器）是 DRAM 的 3D 堆叠变体。
- **SRAM**（静态随机存取存储器）：6 个晶体管构成双稳态电路，速度快但面积大。用作缓存（Cache）。

8.4 缓存（Cache）

缓存是 CPU 和主存之间的高速小容量缓冲。现代 CPU 通常有 L1、L2、L3 三级缓存。

8.4.1 直接映射缓存（Direct-Mapped）

$$\text{缓存行索引} = (\text{块地址}) \bmod (\text{缓存行数}) \quad (24)$$

每个内存块只能映射到唯一一个缓存位置。硬件简单但冲突缺失多。

8.4.2 组相联缓存 (Set-Associative)

$$\text{组索引} = (\text{块地址}) \bmod (\text{组数}) \quad (25)$$

每组有多路（通常 2-16 路）。是直接映射与全相联的折中。

8.4.3 缓存地址划分

$$\text{地址} = [\text{Tag} \mid \text{Index} \mid \text{Block Offset}]$$

- Block Offset: 块内字节偏移
- Index: 选择缓存行（直接映射）或缓存组（组相联）
- Tag: 验证该缓存行是否存的是所需数据

8.4.4 缓存缺失与写策略

表 12: 缓存写策略

策略	写命中	写缺失
写穿透 (Write-Through)	同时更新缓存和主存	直接写主存
写回 (Write-Back)	只更新缓存, 标记 dirty	先分配缓存行, 再写入
分配 (Write-Allocate)	—	分配缓存行 (常配合写回)
非分配 (No-Write-Allocate)	—	不分配, 直接写主存

8.4.5 缓存性能

$$\text{平均访存时间} = \text{命中时间} + \text{缺失率} \times \text{缺失代价}$$

$$\text{AMAT} = T_{\text{hit}} + \text{Miss Rate} \times \text{Miss Penalty}$$

8.5 虚拟内存

虚拟内存使每个进程拥有独立的地址空间，且程序可用的地址可以超过物理内存容量。

定义 8.3 (虚拟地址与物理地址).

$$\text{虚拟地址} \xrightarrow{MMU + TLB} \text{物理地址}$$

MMU (*Memory Management Unit*) 负责转换, 页表 (*Page Table*) 存储映射关系。

8.5.1 TLB (快表)

TLB (Translation Lookaside Buffer) 是页表的专用缓存, 缓存最近使用的虚拟 $\rightarrow$ 物理地址映射。TLB 缺失的代价非常高 (需要走访页表, 甚至引起缺页中断)。

8.5.2 多级页表

现代操作系统使用多级页表减少页表本身的内存开销 (如 x86-64 的 4 级页表)。

8.6 存储层次设计原则

- 越靠近 CPU 的层次, 容量越小、速度越快、单价越贵
- 包含性 (Inclusion): L1 数据也在 L2 和 L3 中 (多数实现)
- 排他性 (Exclusion): 不同层次不重叠数据 (AMD 的缓存策略)
- 一致性 (Coherence): 多核共享数据时保证所有核看到的值一致

9 I/O 系统与总线

9.1 I/O 系统的层次

输入/输出 (I/O) 系统负责 CPU 与外部世界的通信:

- 设备层: 键盘、鼠标、显示器、硬盘、网卡、GPU
- 设备控制器: 硬件接口, 连接设备和总线
- 设备驱动: 操作系统软件, 向 OS 提供统一接口
- I/O 指令 / 内存映射 I/O: CPU 如何与控制器交互

9.2 内存映射 I/O vs 独立 I/O

表 13: 两种 I/O 模型

特征	内存映射 I/O	独立 I/O
地址空间	I/O 设备与内存共用地址	I/O 有独立地址空间
访问方式	lw/sw 指令	专用 IN/OUT 指令
代表	MIPS、ARM、RISC-V	x86 (in / out)
设计	简单, 无需额外指令	地址不冲突, 但增加复杂度

9.3 I/O 同步方式

9.3.1 轮询 (Polling)

CPU 反复读取设备状态寄存器, 直到设备就绪。简单但浪费 CPU 时间。

```
while (status_reg != READY) { /* 等待 */ };
```

9.3.2 中断 (Interrupt)

设备就绪时向 CPU 发出中断信号。CPU 暂停当前执行, 跳转到中断服务程序 (ISR), 处理完毕后恢复。

- 中断向量表: 存储各中断号对应的 ISR 地址
- 中断优先级: 高优先级中断可打断低优先级
- 可屏蔽中断 vs 不可屏蔽中断 (NMI)

9.3.3 直接存储器访问 (DMA)

DMA 控制器直接控制总线，在设备和内存之间搬移数据，不经过 CPU。适用于大批量数据传输（如磁盘读写、网络包）。

- CPU 配置 DMA 控制器（源地址、目标地址、传输大小）
- DMA 独立完成传输
- 完成后向 CPU 发中断通知

9.4 总线

总线 (Bus) 是连接 CPU、内存和 I/O 设备的共享通信通道。

9.4.1 总线分类

表 14: 总线类型

总线	连接对象	代表
处理器-内存总线	CPU ↔ 内存	前-总线 (FSB)、QPI、Infinity Fabric
I/O 总线	CPU ↔ 外围设备	PCI、PCIe、USB
背板总线	模块间互连	VME、CompactPCI

9.4.2 现代总线：PCIe

PCI Express (PCIe) 取代了传统并行总线 (PCI)，采用高速串行点对点连接：

- **Lane**: 每 Lane 由一对差分信号（发送 + 接收）组成
- **带宽**: PCIe 3.0 $\times 1 \approx 1$ GB/s, $\times 16 \approx 16$ GB/s
- **拓扑**: Root Complex → Switch → Endpoint, 树形结构
- 热插拔、链路协商、错误检测与重传

9.5 磁盘与固态存储

- **HDD (机械硬盘)**: 旋转盘片 + 磁头臂, 寻道时间 + 旋转延迟 + 传输时间
- **SSD (固态硬盘)**: NAND Flash, 无机械部件, 随机读快、顺序读写快
- **NVMe**: 基于 PCIe 的快速存储协议 (取代 SATA/AHCI), 延迟 $< 10 \mu s$

9.6 多核与并行 I/O

现代多核系统面临 I/O 瓶颈：

- **DMA 竞争**：多个核心共享 DMA 通道和内存带宽
- **IOMMU**：用类似 MMU 的机制保护设备对内存的访问
- **RDMA**：远程直接内存访问，跨网络访问远端内存（InfiniBand、RoCE）
- **DPDK / SPDK**：绕过内核，用户态直接操作 I/O，降低延迟

9.7 系统总线架构总结

现代计算机的总线结构已经从” 共享总线” 演变为” 互连网络”：

$\text{CPU} \xleftrightarrow{\text{QPI} / \text{UPI} / \text{IF}} \text{内存控制器} \rightarrow \text{DDR5} \quad ; \quad \text{CPU} \xleftrightarrow{\text{PCIe}} \text{GPU} / \text{NVMe} / \text{NIC}$

这种点对点互联取代了传统共享总线，提供了更好的带宽隔离和可扩展性。